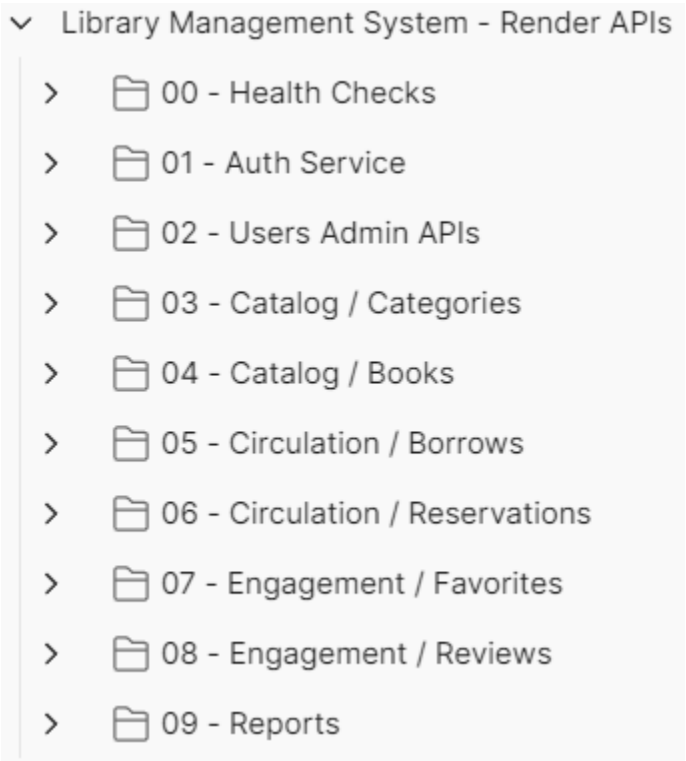
Library Management System – Evidence & Requirements Documentation

1. Implementation (APIs)

API Collection Structure

The following screenshot shows the organized Postman collection used for testing all system APIs across all microservices.

- Health Checks
- Authentication Service
- Users / Admin APIs
- Catalog APIs
- Circulation APIs
- Engagement APIs
- Reports APIs



API Collection Structure

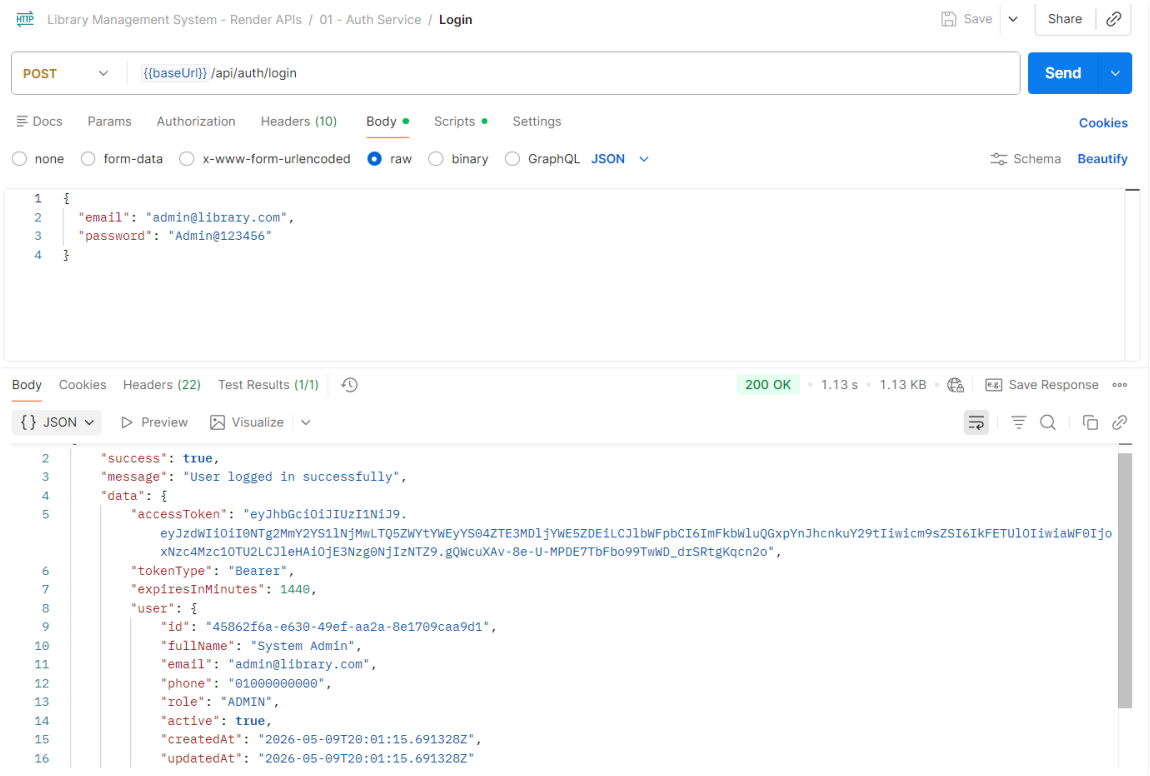
Authentication – Login API

Endpoint

`POST /api/auth/login`

Features Demonstrated

- JWT Authentication
- Secure Login
- Role-based Authorization
- Token Generation



Login API

Users Administration – Get Users

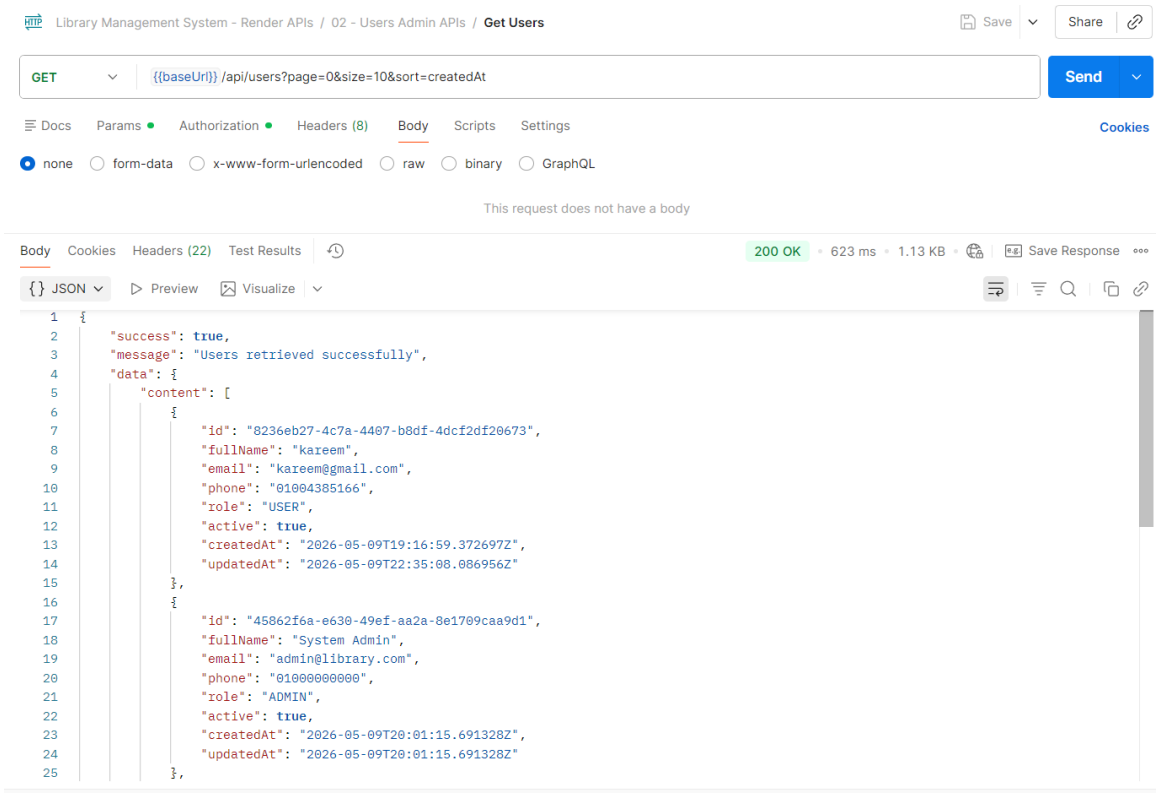
Endpoint

`GET /api/users`

Features Demonstrated

- Admin Authorization
- Pagination

- Secure Protected Endpoints



Get Users

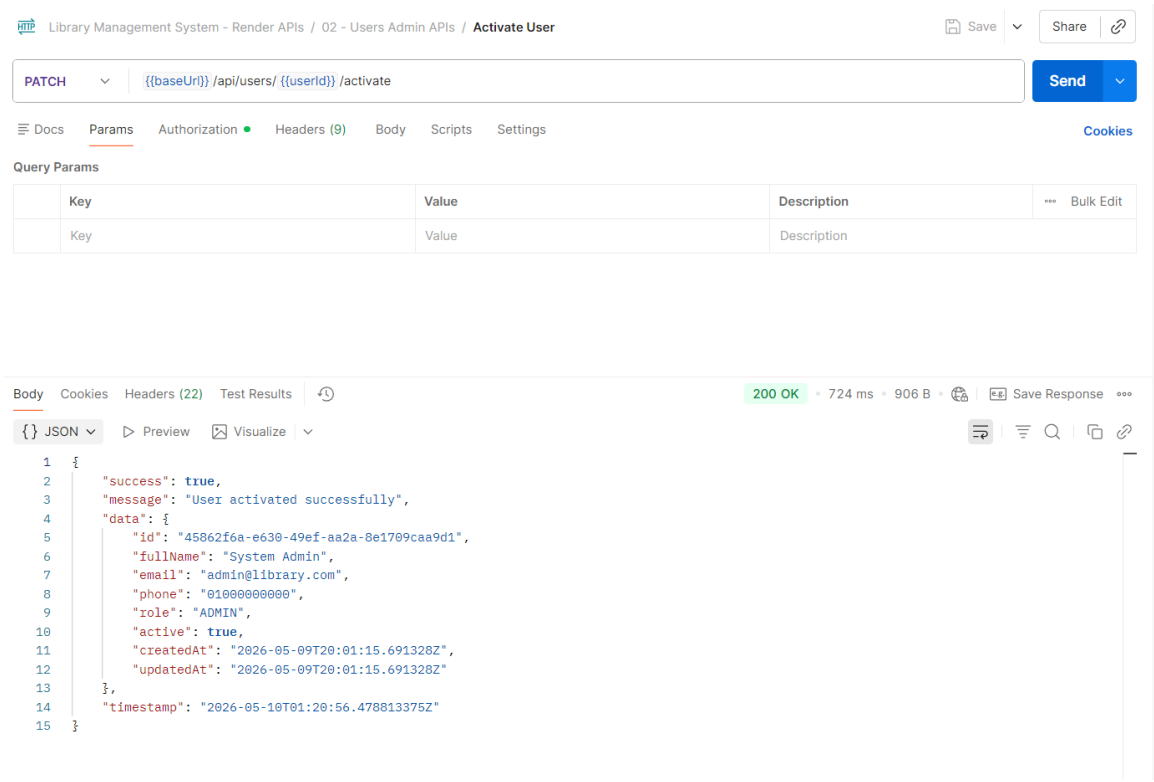
Users Administration – Activate User

Endpoint

`PATCH /api/users/{id}/activate`

Features Demonstrated

- Role-Based Access Control
- Account Management
- Update Operations



Activate User

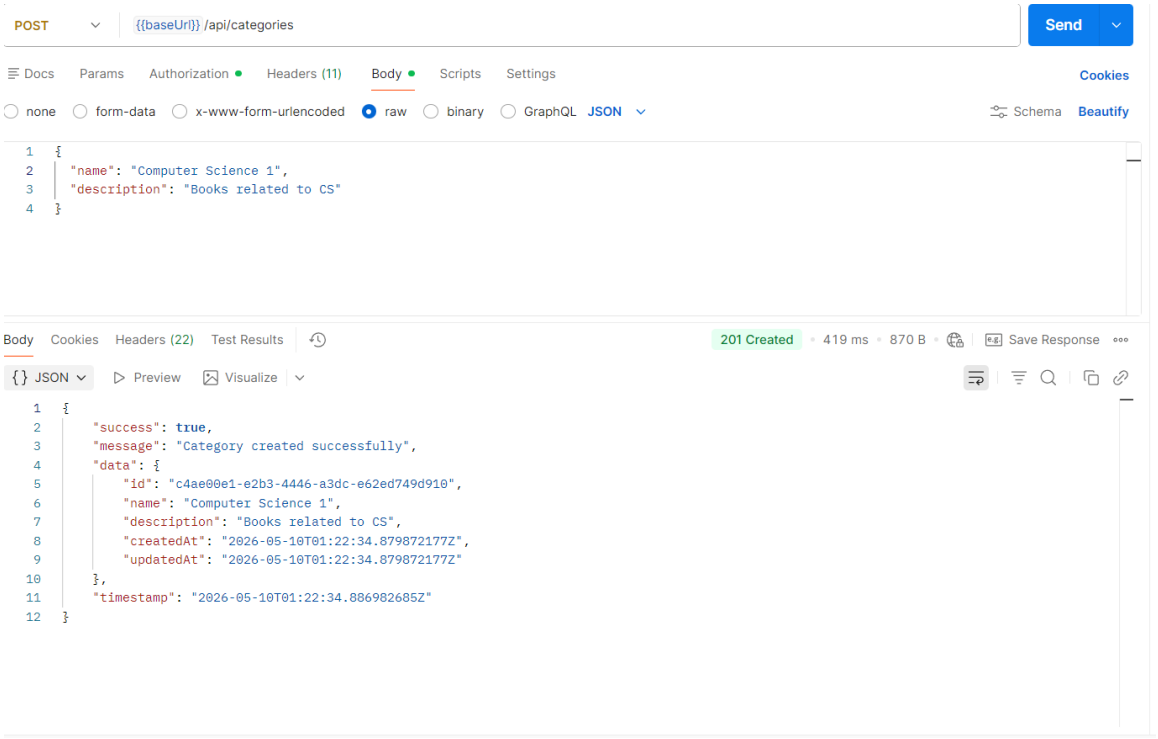
Catalog Service – Create Category

Endpoint

`POST /api/categories`

Features Demonstrated

- CRUD Operations
- Validation
- Catalog Management



Create Category

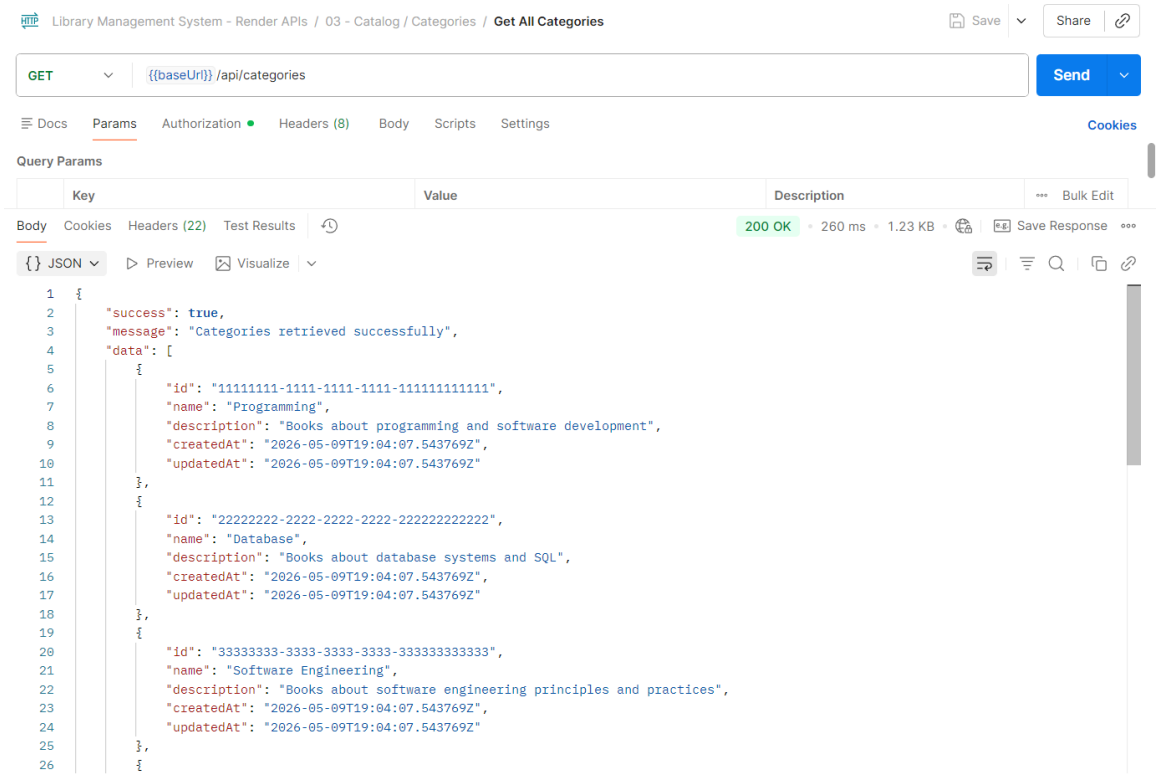
Catalog Service – Get Categories

Endpoint

`GET /api/categories`

Features Demonstrated

- Data Retrieval
- API Pagination Support
- Secure Service Communication



Get Categories

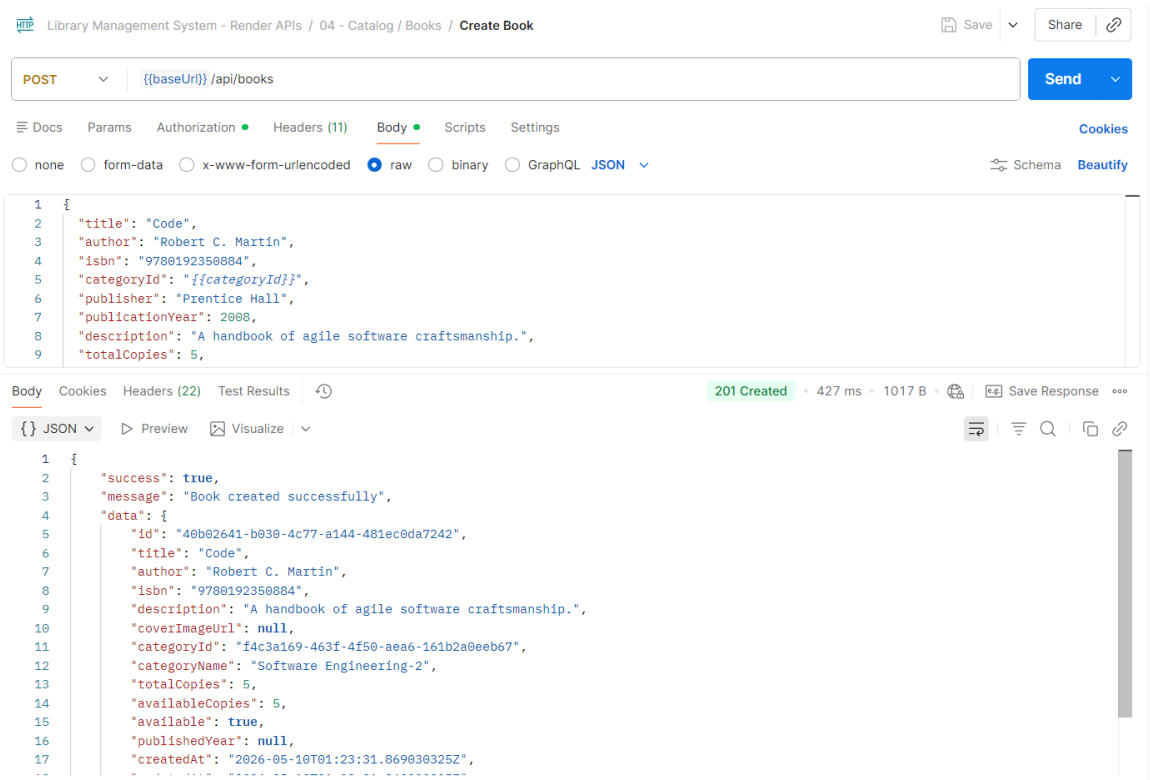
Catalog Service – Create Book

Endpoint

`POST /api/books`

Features Demonstrated

- DTO Usage
- Relationship Mapping
- Business Validation



Create Book

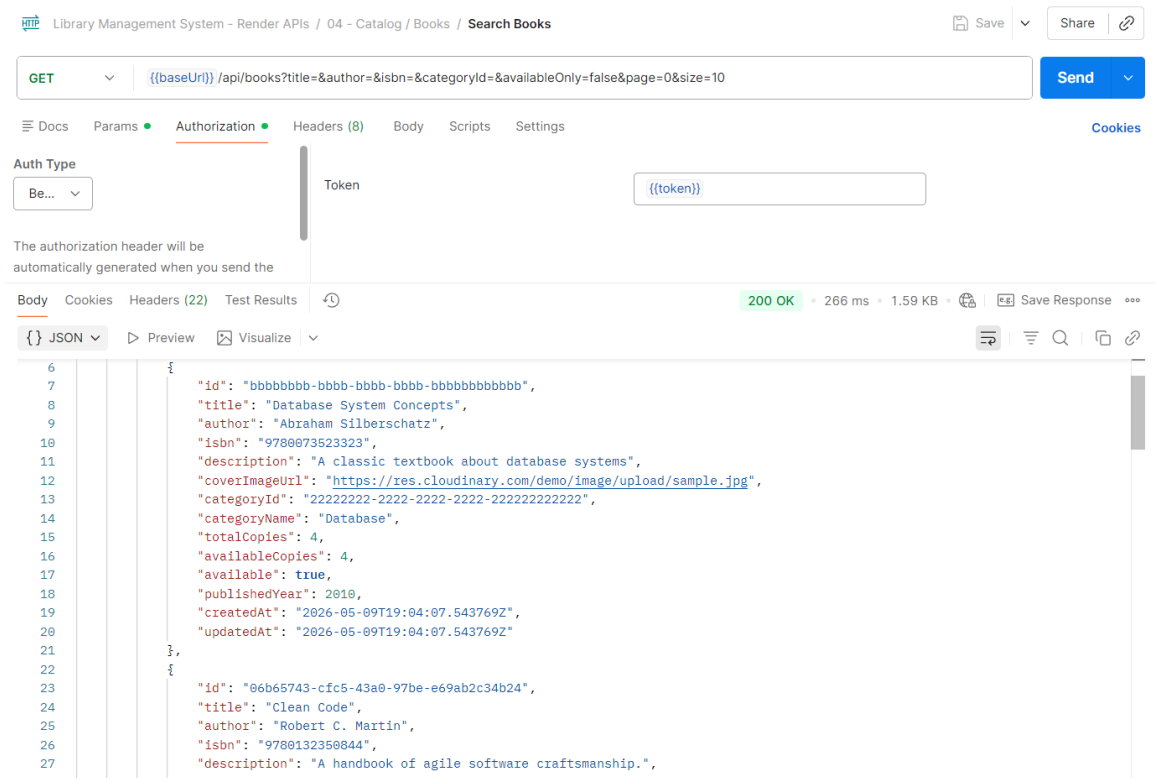
Catalog Service – Search Books

Endpoint

`GET /api/books`

Features Demonstrated

- Filtering
- Search Functionality
- Availability Checks



Search Books

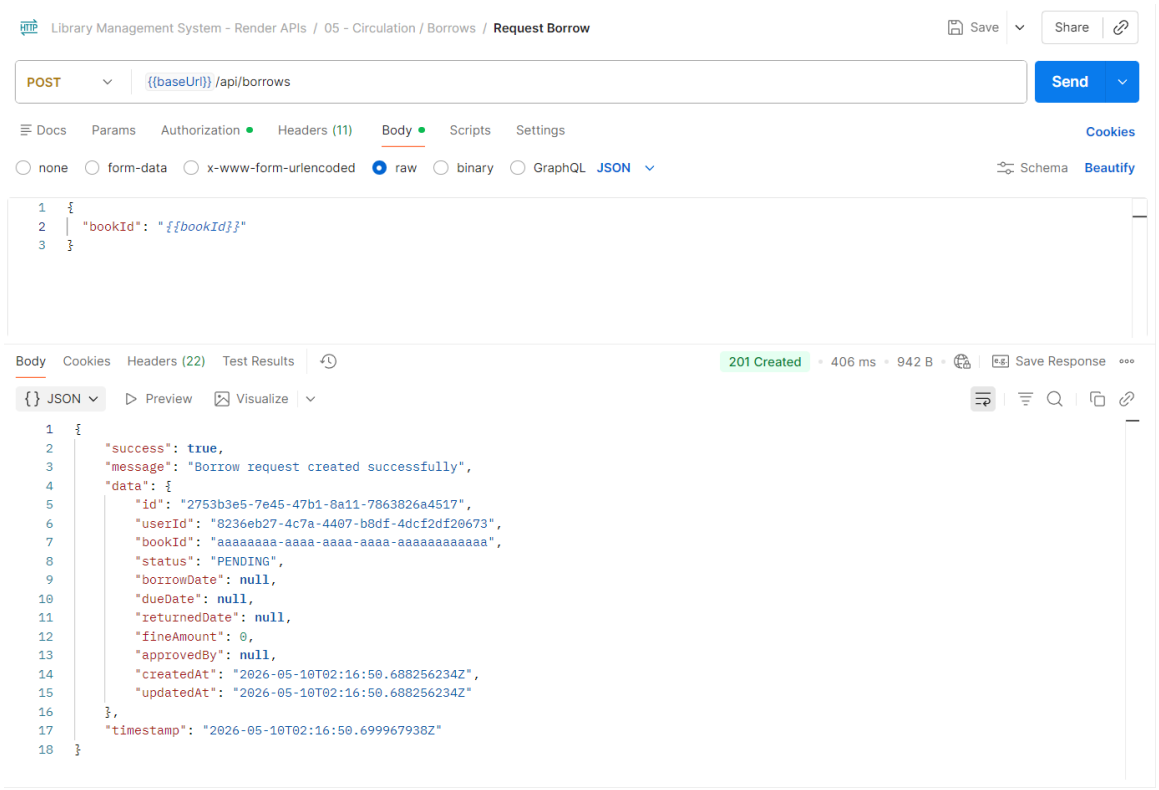
Circulation Service – Borrow Request

Endpoint

`POST /api/borrows`

Features Demonstrated

- Borrow Workflow
- Book Availability Logic
- Request Status Handling



Borrow Request

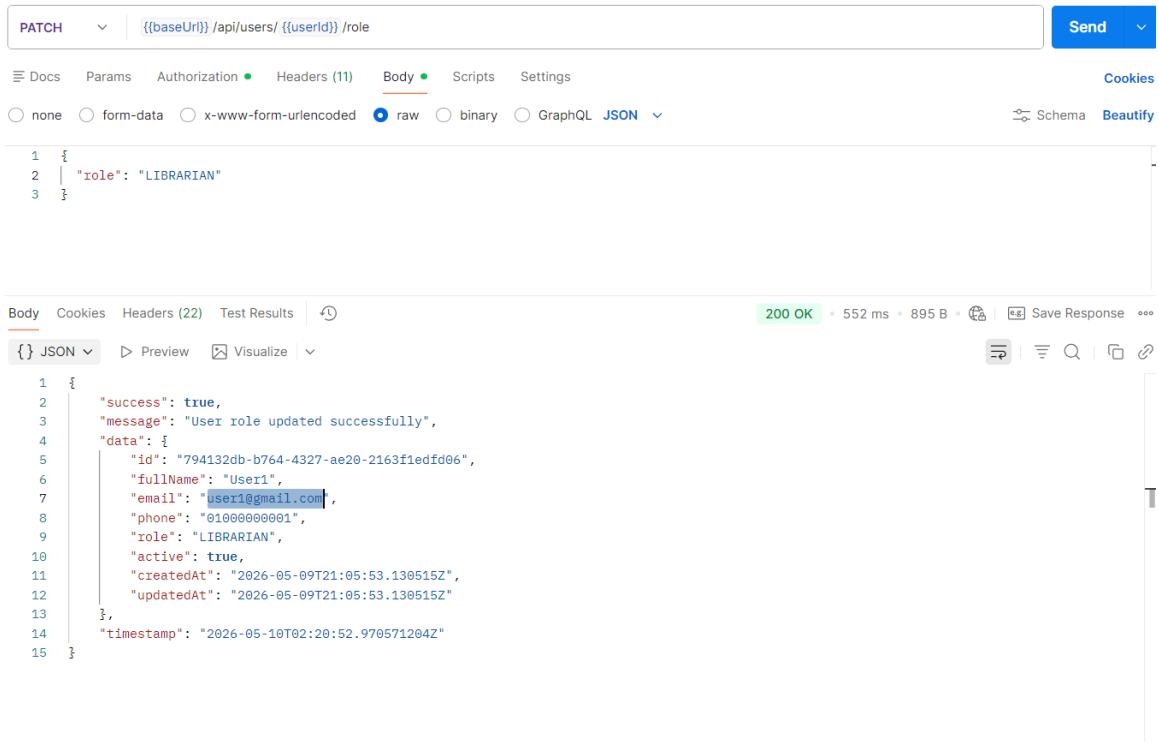
Users Administration – Update Role

Endpoint

`PATCH /api/users/{id}/role`

Features Demonstrated

- RBAC Authorization
- Dynamic Role Management



Update Role

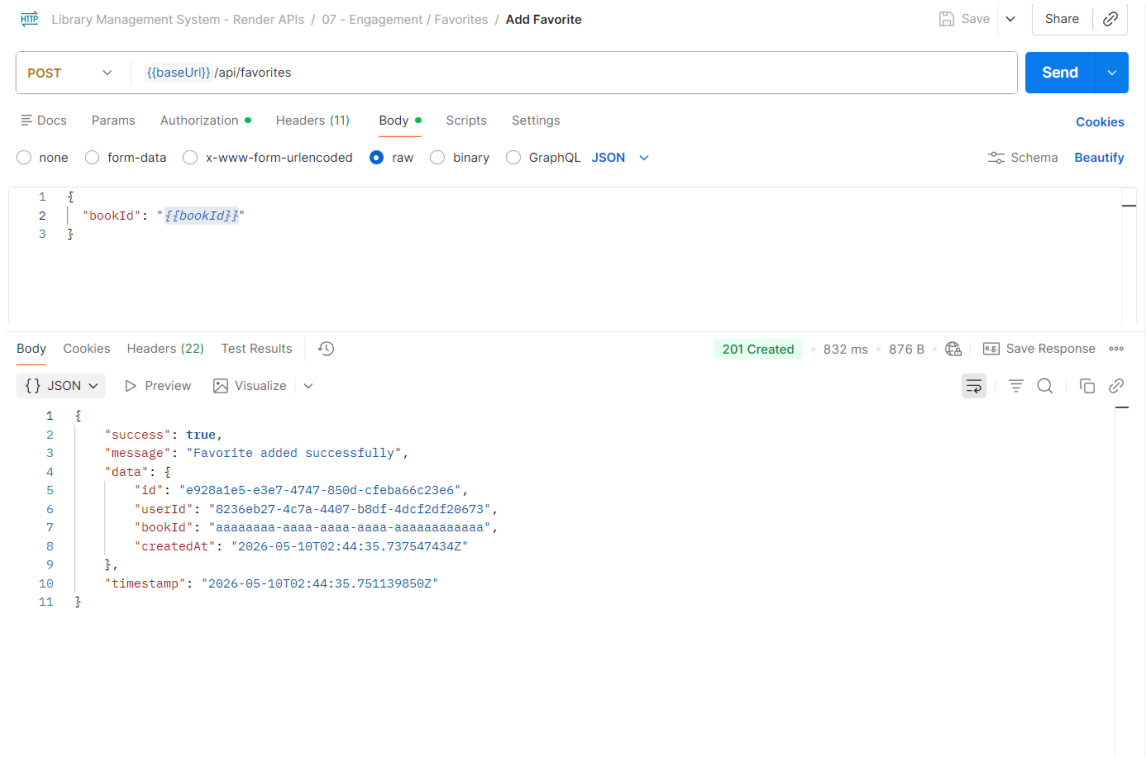
Engagement Service – Add Favorite

Endpoint

`POST /api/favorites`

Features Demonstrated

- User Interaction
- Personalized User Features



Add Favorite

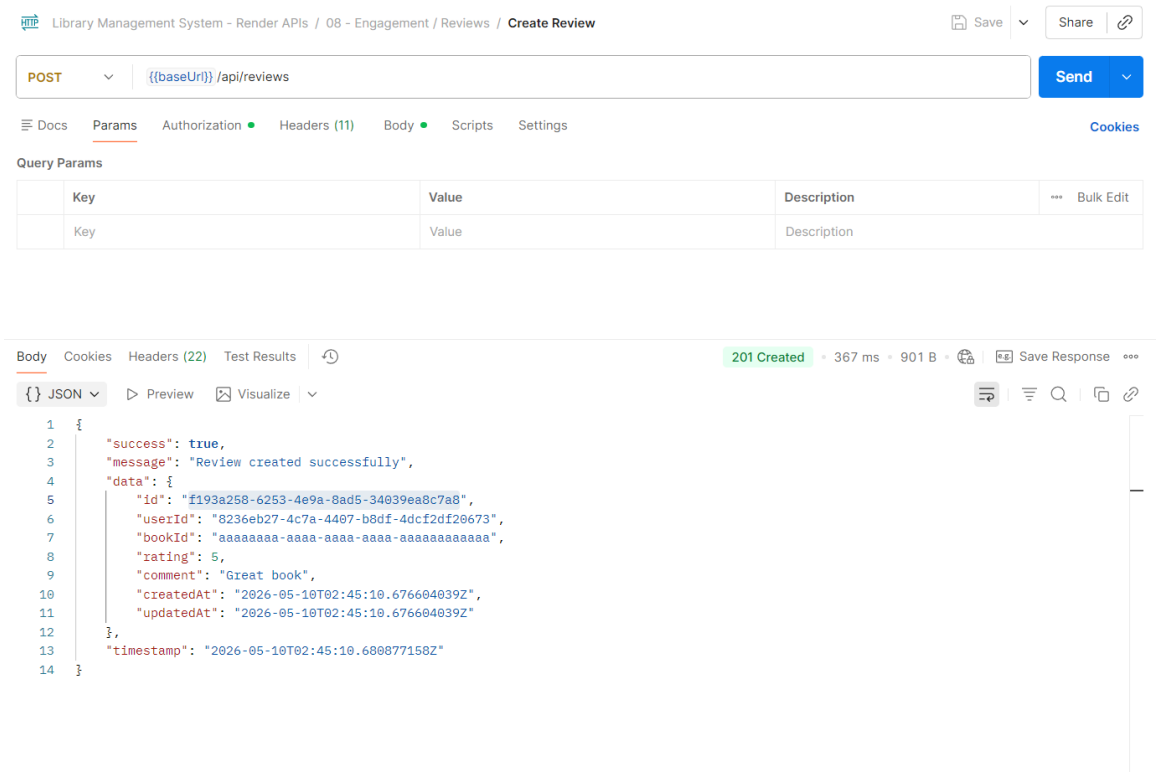
Engagement Service – Create Review

Endpoint

`POST /api/reviews`

Features Demonstrated

- Reviews System
- Ratings System
- User Feedback Management



Create Review

---

2. Object Constraint Language (OCL)

The project applies business constraints and validation rules similar to OCL concepts.

Example Constraints

```
context User
inv UniqueEmail:
User.allInstances()->isUnique(email)

context Borrow
inv BorrowAvailableBook:
book.availableCopies > 0

context Review
inv ValidRating:
rating >= 1 and rating <= 5

context Book
inv PositiveCopies:
totalCopies > 0
```

OCL Validation from Real Code

The following screenshot demonstrates validation and business rules implemented inside the service layer.

Constraints Applied

- Email normalization
- Duplicate email prevention
- Transaction management
- Business validation

---

3. Aspect Oriented Programming (AOP)

The project uses Aspect-Oriented Programming concepts for handling cross-cutting concerns.

AOP Features Used

- Logging
- Monitoring
- Exception Tracking
- Proxy-Based Interception

AOP Concept

Instead of repeating logging code inside every service method:

```
System.out.println("Method Started");
```

Spring AOP intercepts methods automatically using proxies and aspects.

Benefits

- Separation of Concerns
- Cleaner Code
- Reusability
- Centralized Logging

---

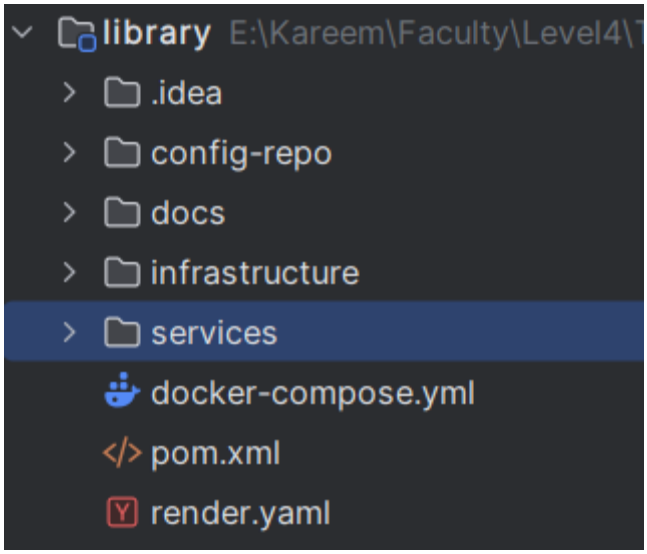
4. Docker

The entire system is dockerized.

Dockerized Components

- PostgreSQL
- Config Server
- Discovery Server
- API Gateway
- All Microservices

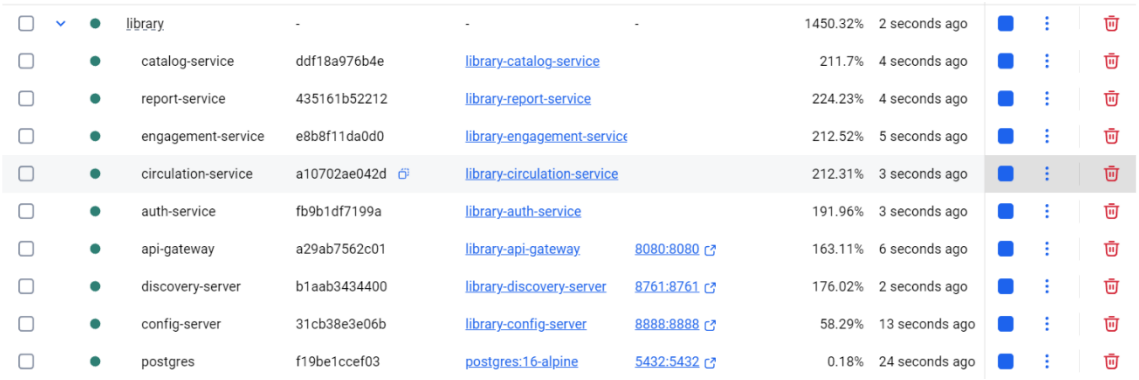
Project Infrastructure Structure



Project Structure

Running Docker Containers

The following screenshot demonstrates all running containers successfully.



library	-	-	-		1450.32%	2 seconds ago	
catalog-service	ddf18a976b4e	library-catalog-service			211.7%	4 seconds ago	
report-service	435161b52212	library-report-service			224.23%	4 seconds ago	
engagement-service	e8b8f11da0d0	library-engagement-service			212.52%	5 seconds ago	
circulation-service	a10702ae042d	library-circulation-service			212.31%	3 seconds ago	
auth-service	fb9b1df7199a	library-auth-service			191.96%	3 seconds ago	
api-gateway	a29ab7562c01	library-api-gateway	8080:8080		163.11%	6 seconds ago	
discovery-server	b1aab3434400	library-discovery-server	8761:8761		176.02%	2 seconds ago	
config-server	31cb38e3e06b	library-config-server	8888:8888		58.29%	13 seconds ago	
postgres	f19be1ccef03	postgres:16-alpine	5432:5432		0.18%	24 seconds ago	

Docker Containers

---

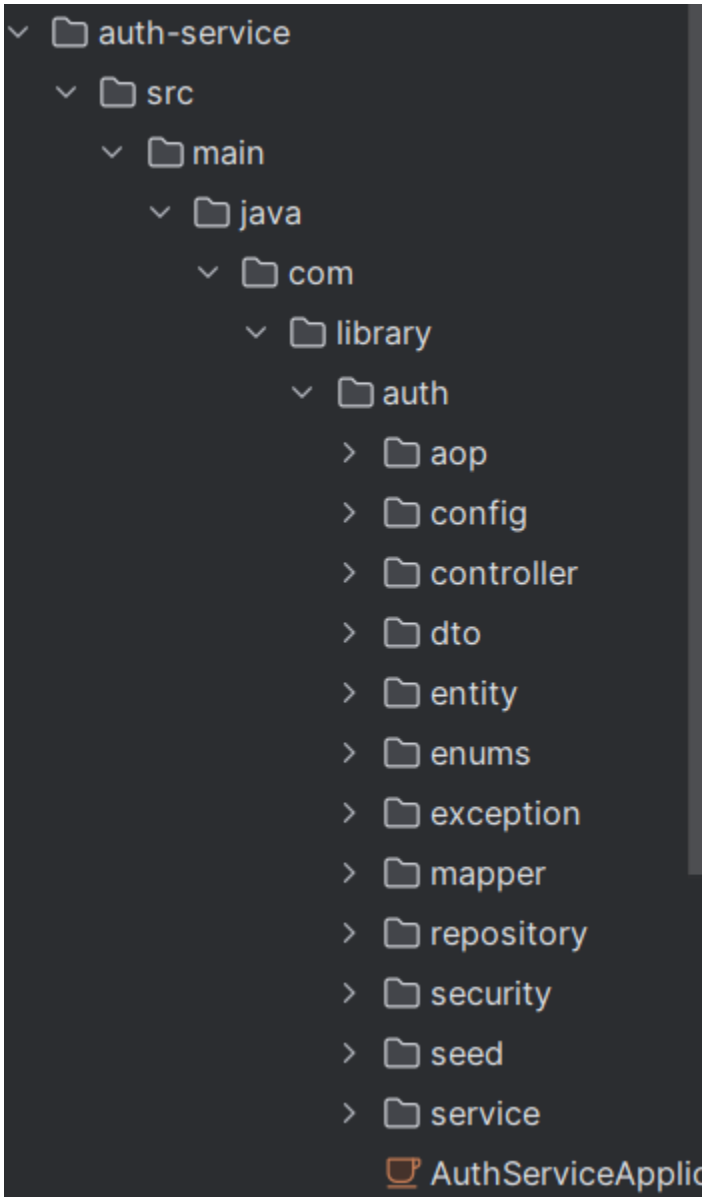
5. Clean Code

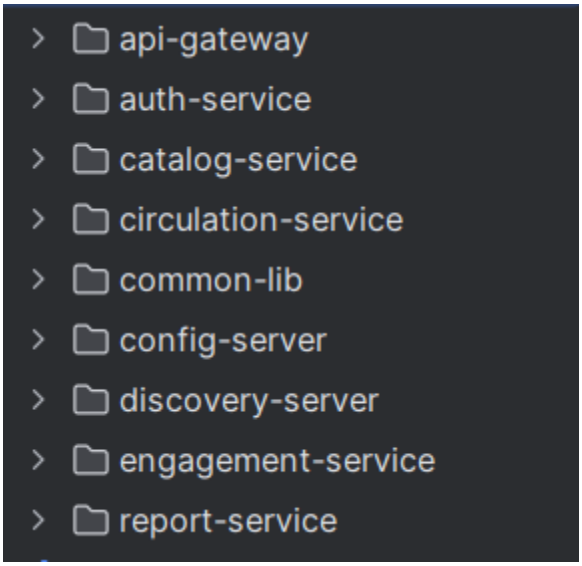
The project follows clean architecture and clean code principles.

Package Organization

- controller
- service
- repository
- dto
- mapper
- config
- security
- exception

Organized Project Structure





Clean Architecture

Clean Code Principles Applied

Separation of Concerns

Each layer has a clear responsibility.

Constructor Injection

Dependencies are injected using constructors.

DTO Pattern

DTOs are separated from entities.

Naming Conventions

Meaningful names are used for methods and classes.

---

6. Design Patterns

The project implements multiple design patterns.

---

1. Singleton Pattern

Usage

Spring Beans are singleton by default.

Code Example

```
@Service
public class AuthServiceImpl implements AuthService
```

Benefit

- Shared instance
- Better memory management
- Centralized service logic

---

2. Repository Pattern

Usage

Repositories abstract database access.

Code Example

```
public interface UserRepository extends JpaRepository<User, UUID>
```

Benefit

- Cleaner data access
- Easier maintenance
- Better abstraction

---

3. DTO Pattern

Usage

DTOs are used between APIs and services.

Examples

- LoginRequest
- RegisterRequest
- AuthResponse

Benefit

- Secure API contracts

- Prevent exposing entities

---

4. Builder Pattern

Usage

Objects are created using builders.

Example

```
AuthResponse.builder()
```

Benefit

- Cleaner object creation
- Readable code

---

5. Proxy Pattern (AOP)

Usage

Spring AOP creates proxy objects around services.

Benefit

- Logging without modifying business logic
- Centralized monitoring

---

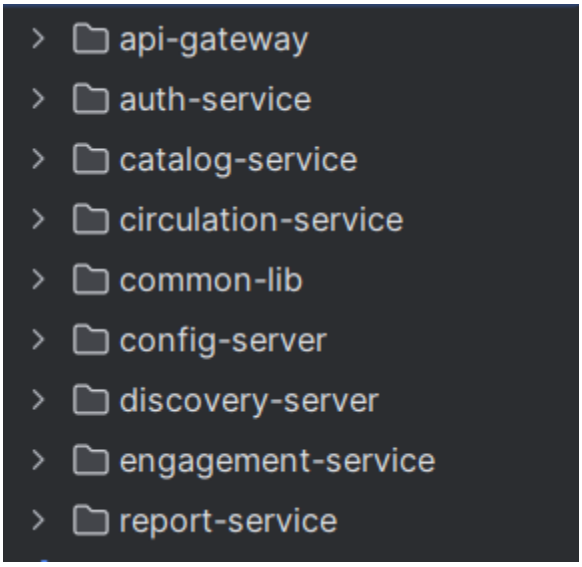
7. Microservices & Cloud

The project is fully implemented using Microservices Architecture.

Implemented Services

- API Gateway
- Config Server
- Discovery Server
- Auth Service
- Catalog Service
- Circulation Service
- Engagement Service
- Report Service

Services Structure



Microservices Structure

Eureka Discovery Server

The following screenshot demonstrates all services registered successfully in Eureka.

Instances currently registered with Eureka

Application	AMIs	Availability Zones	Status
API-GATEWAY	n/a (1)	(1)	UP (1) - a29ab7562c01:api-gateway:8080
AUTH-SERVICE	n/a (1)	(1)	UP (1) - fb9b1df7199a:auth-service:8081
CATALOG-SERVICE	n/a (1)	(1)	UP (1) - ddf18a976b4e:catalog-service:8082
CIRCULATION-SERVICE	n/a (1)	(1)	UP (1) - a10702ae042d:circulation-service:8083
ENGAGEMENT-SERVICE	n/a (1)	(1)	UP (1) - e8b8f11da0d0:engagement-service:8084
REPORT-SERVICE	n/a (1)	(1)	UP (1) - 435161b52212:report-service:8085

Eureka Dashboard

Cloud Deployment

The following screenshot demonstrates successful deployment and running services.

SERVICE NAME 9	STATUS	RUNTIME	REGION	UPDATED ↓
library-circulation-service	✓ Deployed	Docker	Oregon	29min
library-api-gateway	✓ Deployed	Docker	Oregon	50min
library-auth-service	✓ Deployed	Docker	Oregon	5h
library-engagement-service	✓ Deployed	Docker	Oregon	5h
library-catalog-service	✓ Deployed	Docker	Oregon	5h
library-report-service	✓ Deployed	Docker	Oregon	5h
library-discovery-server	✓ Deployed	Docker	Oregon	5h
library-config-server	✓ Deployed	Docker	Oregon	5h
library-postgres	✓ Available	PostgreSQL 18	Oregon	21h

Cloud Deployment

Conclusion

The Library Management System successfully demonstrates:

- RESTful APIs
- JWT Authentication
- Role-Based Access Control
- Dockerized Infrastructure
- Eureka Service Discovery
- API Gateway Routing
- Clean Architecture
- Design Patterns
- AOP Concepts
- OCL-style Constraints
- Cloud Deployment

The project is production-ready and fully deployed using Docker containers and cloud infrastructure.